

UNIT IV NEURAL NETWORKS

Multilayer perceptron, activation functions, network training – gradient descent optimization – stochastic gradient descent, error backpropagation, from shallow networks to deep networks – Unit saturation (aka the vanishing gradient problem) – ReLU, hyperparameter tuning, batch normalization, regularization, dropout.

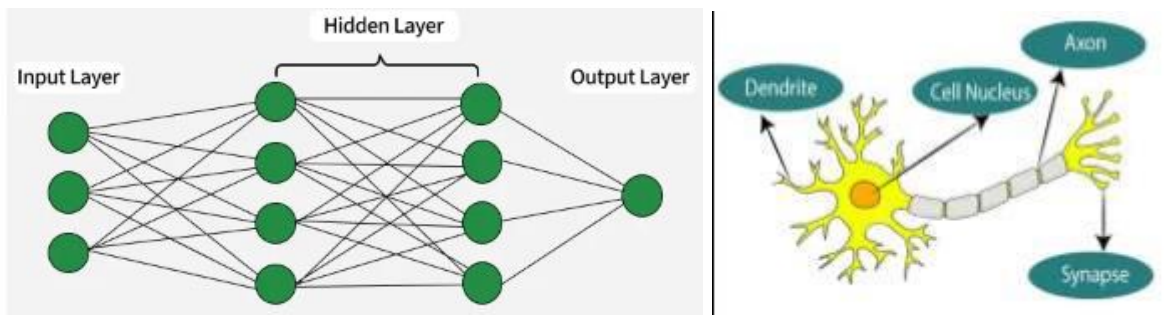
INTRODUCTION TO NEURAL NETWORKS

Neural networks are machine learning models that mimic the complex functions of the human brain. These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making.

Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules. These networks are built from several key components:

- **Neurons:** The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- **Connections:** Links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases:** These parameters determine the strength and influence of connections.
- **Propagation Functions:** Mechanisms that help process and transfer data across layers of neurons.
- **Learning Rule:** The method that adjusts weights and biases over time to improve accuracy.

Layers in Neural Network Architecture



1. **Input Layer:** This is where the network receives its input data. Each input neuron in the layer corresponds to a feature in the input data.
2. **Hidden Layers:** These layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each layer consists of units (neurons) that transform the inputs into something that the output layer can use.
3. **Output Layer:** The final layer produces the output of the model. The format of these outputs varies depending on the specific task like classification, regression.

Working of Neural Networks/Multilayer Perceptron

1. Forward Propagation

In forward propagation the data flows from the input layer to the output layer, passing through any hidden layers. Each neuron in the hidden layers processes the input as follows:

1. Weighted Sum: The neuron computes the weighted sum of the inputs:

$$z = \sum_i w_i x_i + b$$

Where:

- x_i is the input feature.
- w_i is the corresponding weight.
- b is the bias term.

2. Activation Function: The weighted sum z is passed through an activation function to introduce non-linearity. Common activation functions include:

- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
- ReLU (Rectified Linear Unit): $f(z) = \max(0, z)$
- Tanh (Hyperbolic Tangent): $\tanh(z) = \frac{2}{1+e^{-2z}} - 1$

2. Loss Function

Once the network generates an output the next step is to calculate the loss using a [loss function](#). In supervised learning this compares the predicted output to the actual label.

For a classification problem the commonly used [binary cross-entropy](#) loss function is:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Where:

- y_i is the actual label.
- $\hat{y}_i$ is the predicted label.
- N is the number of samples.

For regression problems the [mean squared error \(MSE\)](#) is often used:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

3. Backpropagation

The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through [backpropagation](#):

1. **Gradient Calculation:** The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
2. **Error Propagation:** The error is propagated back through the network, layer by layer.
3. **Gradient Descent:** The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss: $w = w - \eta \cdot \frac{\partial L}{\partial w}$

Where:

- w is the weight.
- η is the learning rate.
- $\frac{\partial L}{\partial w}$ is the gradient of the loss function with respect to the weight.

4. Optimization

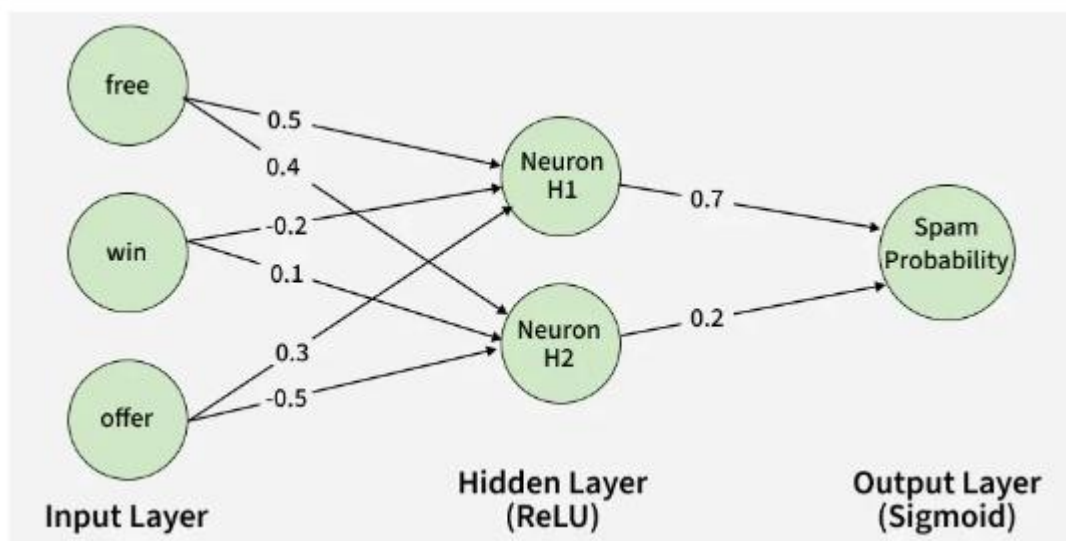
MLPs rely on optimization algorithms to iteratively refine the weights and biases during training. Popular optimization methods include:

- **Stochastic Gradient Descent (SGD):** Updates the weights based on a single sample or a small batch of data: $w = w - \eta \cdot \frac{\partial L}{\partial w}$
- **Adam Optimizer:** An extension of SGD that incorporates momentum and adaptive learning rates for more efficient training:

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \cdot g_t$
- $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \cdot g_t^2$

Here g_t represents the gradient at time t and β_1, β_2 are decay rates.

EXAMPLE



Types of Neural Networks

There are seven types of neural networks that can be used.

- **Feedforward Networks:** It is a simple artificial neural network architecture in which data moves from input to output in a single direction.
- **Singlelayer Perceptron:** It has one layer and it applies weights, sums inputs and uses activation to produce output.
- **Multilayer Perceptron (MLP):** It is a type of feedforward neural network with three or more layers, including an input layer, one or more hidden layers and an output layer. It uses nonlinear activation functions.
- **Convolutional Neural Network (CNN):** It is designed for image processing. It uses convolutional layers to automatically learn features from input images, enabling effective image recognition and classification.
- **Recurrent Neural Network (RNN):** Handles sequential data using feedback loops to retain context over time.
- **Long Short-Term Memory (LSTM):** A type of RNN with memory cells and gates to handle long-term dependencies and avoid vanishing gradients.

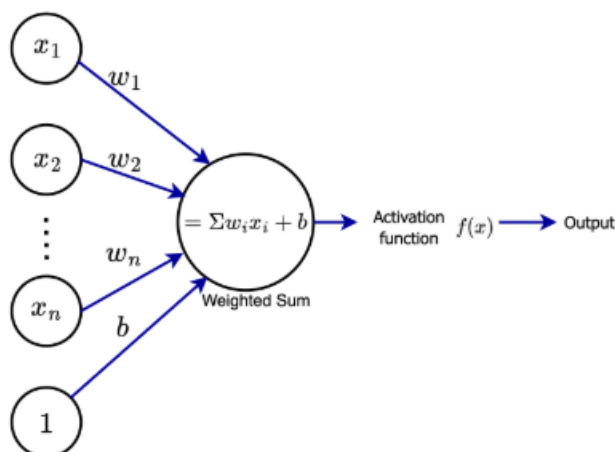
ACTIVATION FUNCTIONS

Activation functions are **the heartbeat of neural networks**. They introduce **non-linearity**, enabling networks to model complex patterns that linear models cannot. **An Activation Function** decides whether a neuron should be activate or not.

Without activation functions:

- A multi-layer network collapses into a **linear function**, no matter how many layers it has.
- It cannot learn complex mappings like image recognition, language understanding, or time-series forecasting.

Choosing the right activation function can **drastically affect training speed, stability, and accuracy**.



Why Activation Functions Matter

An activation function transforms the input z of a neuron into an output a :

Roles:

1. Introduce **non-linearity**.
2. Control **range of outputs** (probabilities, zero-centered).
3. Affect **gradient flow** during backpropagation.

Popular Activation Functions

a. ReLU (Rectified Linear Unit)

- Non-linear, computationally cheap.
- Solves vanishing gradient problem for positive inputs.
- Sparse activation: many zeros $\rightarrow$ efficiency.

B. Sigmoid

- Maps to **0–1**, good for probabilities.
- Early hidden layers or binary outputs.

C. Tanh (Hyperbolic Tangent)

- Maps to **-1 to 1**, zero-centered.
- Better than sigmoid for hidden layers.

D. GELU (Gaussian Error Linear Unit)

- Smooth, probabilistic version of ReLU.
- Popular in **transformers** (BERT, GPT).